LARGE LANGUAGE MODELS

Complete Study Notes

Foundations, Tokens, Context, Behavior & Parameters

Module 1	**LLM Intuition**	75 min
Module 2	**Tokens Deep Dive**	75 min
Module 3	**Context Window**	75 min
Module 4	**LLM Behavior & Limitations**	45 min
Module 5	**LLM Parameters (Temp, Top-P, etc.)**	60 min
Module 6	**Hands-on Integration**	45 min

Day 1 — Before the API

MODULE 1 — LLM Intuition (Foundation First)

Duration: ~75 minutes | Teaching Style: Explain → Show → Practice

1. What is an LLM?

A Large Language Model (LLM) is fundamentally a next-token predictor. This is the single most important concept to understand before anything else.

Core Definition: LLM = Next Token Predictor. Nothing more, nothing less at its foundation.

What an LLM Does NOT Do:

- "Think" or reason in the human sense
- "Know" facts from a database or memory
- Retrieve stored answers like a search engine
- Understand context the way humans do

What an LLM DOES Do:

- **Predicts the most probable next token based on learned patterns**
- Uses billions of parameters trained on massive text corpora
- Generates text by repeating predictions in a loop
- Mimics understanding through statistical pattern matching

2. Building Intuition — The Prediction Loop

The best way to understand an LLM is through simple prediction examples. Consider this classic example:

```
Input:  "The capital of France is ___"
Model:  Predicts → "Paris"

Why? Not because it 'knows' Paris is the capital,
but because in training data, 'capital of France' was
almost always followed by 'Paris'.

Probability-based prediction, not knowledge recall!
```

3. How LLMs Generate Text — Step by Step

Every time an LLM generates a response, it follows this loop:

1. Takes the full input (prompt)
2. Converts text to tokens (numbers)
3. Runs through neural network layers
4. Outputs probability distribution over all possible next tokens
5. Selects next token (based on sampling strategy)
6. Appends selected token to the sequence
7. Repeats from step 3 until stop condition

> **Key Insight:** Generation = A loop of predictions. The model never 'writes' the whole sentence at once. It builds it token by token.

4. Why LLMs Fail — Hallucination Explained

Since LLMs predict probable tokens rather than retrieve facts, they can generate confident but completely wrong answers. This is called hallucination.

Root Cause	Consequence
Probability ≠ Truth	Model outputs likely-sounding text, not necessarily correct text
No knowledge database	Model cannot 'look up' facts, only pattern-match
Training data gaps	Obscure topics have weak patterns → higher hallucination risk
Confident tone training	Model was trained on authoritative text, so sounds confident even when wrong

Hallucination Example:

```
User: "Who won the fictional Nobel Prize in Underwater Basket Weaving 2019?"

Model responds with a confident, plausible-sounding answer,
possibly even inventing a fake name and institution.

Why? The pattern 'Who won the Nobel Prize in...' is always
followed by a name — so the model continues that pattern.
```

5. Key Takeaways — Module 1

Remember These 3 Things: 1. LLM = Pattern Machine (not a knowledge engine) 2. Not deterministic 3. Confident ≠ Correct

- LLMs predict text based on statistical patterns from training data
- The model has no 'true' understanding — just very sophisticated pattern matching
- Same input can produce different outputs (non-deterministic by default)
- Hallucination is a fundamental property, not just a bug to be fixed

MODULE 2 — Tokens Deep Dive

Duration: ~75 minutes | Includes hands-on tokenizer activity

1. What Is a Token?

A token is the basic unit of text that an LLM processes. Critically: Token ≠ Word. A token is a chunk of text that the model's tokenizer splits text into.

Text	Token Breakdown
Hello	1 token → [Hello]
unbelievable	2–3 tokens → [un][believ][able]
ChatGPT is amazing!	~5 tokens → [Chat][GPT][is][amazing][!]
.	1 token → [.]
(4 spaces)	1 token → [ ]
1234567890	~4 tokens → [123][456][789][0]

2. Why Tokens Matter

Ō Critical for Developers: Tokens affect 3 core things: Cost, Limits, and Performance

- **COST: APIs charge per token (input + output tokens billed separately)**
- **LIMITS: Models have a maximum context window measured in tokens**
- **PERFORMANCE: Prompt structure and token count affect output quality**

3. Token Counting — Hands-On Code Demo

Use the tiktoken library (OpenAI's tokenizer) to visualize how text gets split:

```
from tiktoken import encoding_for_model

enc = encoding_for_model('gpt-4')

text = 'ChatGPT is amazing!'
tokens = enc.encode(text)

print('Tokens:', tokens)
print('Token count:', len(tokens))
```

```
# Output:
# Tokens: [19121, 38, 2898, 374, 8056, 0]
# Token count: 5
```

4. Key Observations About Tokenization

- Spaces matter — a space before a word often becomes part of the token
- Symbols are their own tokens (punctuation, brackets, operators)
- Code uses more tokens than prose due to special characters
- Rare words get split into more pieces than common words
- Numbers are often tokenized digit-by-digit or in small groups
- Different models use different tokenizers (GPT-4 vs Claude vs Llama)

5. Token Optimization — Mini Exercise

Exercise: Take the paragraph below and reduce its token count by at least 20% without losing the key meaning.

```
ORIGINAL (high token count):
"Please provide me with a comprehensive and detailed explanation
 of the methodology that you use in order to process and analyze
 the text input that has been provided to you by the user."

OPTIMIZED (lower token count):
"Explain your text processing methodology."

Savings: ~60% fewer tokens, same meaning!
```

6. Token Cost Reference

Developer Note: Always estimate token usage before building production systems. Costs add up quickly at scale.

Text Type	~Tokens per 1K chars	Cost Factor	Notes
Plain English prose	~200–250 tokens	Low	Most efficient
Technical documentation	~250–300 tokens	Medium	More special terms
Source code	~300–400 tokens	Higher	Symbols & structure

Text Type	~Tokens per 1K chars	Cost Factor	Notes
(Python/JS)			
JSON / XML data	~350–450 tokens	High	Lots of brackets/quotes
Minified code	~400–500 tokens	Highest	Dense with symbols

MODULE 3 — Context Window (Core Concept)

Duration: ~75 minutes | 3 hands-on experiments included

1. What Is the Context Window?

The context window is the maximum number of tokens the model can 'see' and process at one time. Think of it as the model's active working memory.

Analogy: Context window = Short-term memory. Fixed size. When full, old information is dropped to make room for new input.

2. What Goes Into the Context Window?

Every token in the following components counts toward your context limit:

- **System message / instructions (set by developer)**
- **All previous conversation turns (user + assistant)**
- **Current user input**
- **Any retrieved documents or injected data (RAG)**
- Function/tool definitions (if using tools)
- The model's own response (output also consumes tokens)

Warning: EVERYTHING counts! Developers often forget that system prompts, chat history, and injected documents all eat into the context limit.

3. What Happens When Context Limit Is Exceeded?

When the input exceeds the model's context window, older messages are dropped (truncated) from the beginning of the conversation.

```
Context Window Visualization:

[ Message 1 (oldest)    DROPPED  ]
[ Message 2              DROPPED  ]
[ Message 3              DROPPED  ]
[ Message 4              KEPT     ]
[ Message 5              KEPT     ]
```

```
[ Message 6              KEPT     ]
[ Current Input          ACTIVE   ]
                       ^
              Context Limit Here
```

4. Context Window Sizes (Reference)

Model	Context Window	~Pages of Text	Best For
GPT-3.5 Turbo	16K tokens	~12 pages	Short tasks
GPT-4o	128K tokens	~96 pages	Long documents
Claude 3.5 Sonnet	200K tokens	~150 pages	Very long docs
Gemini 1.5 Pro	1M tokens	~750 pages	Massive contexts
Llama 3 (8B)	8K tokens	~6 pages	Local/edge use

5. Three Hands-On Experiments

Experiment 1 — Memory Loss Demonstration

8. Tell the model: "My name is Ahmed."
9. Have a long conversation or insert lots of text
10. Eventually ask: "What is my name?"
11. Observe: The model will have forgotten 'Ahmed'

> ‘ **Why It Happened:** The earlier message containing 'My name is Ahmed' was dropped from context when newer messages filled the window.

Experiment 2 — Long Input Test

12. Give the model a very long paragraph (5,000+ words)
13. Ask a question about content at the very beginning
14. Observe: Incomplete understanding, missed details, or truncation
15. This demonstrates 'lost in the middle' phenomenon

Experiment 3 — Token Cost Comparison

16. Ask: "What is 2+2?" with minimal context
17. Ask the same question with 2,000 words of irrelevant background
18. Compare token usage — the second call costs dramatically more
19. Same answer, very different cost

6. Key Teaching Points

ϕ Remember: Bigger context is NOT always better. Manage your context deliberately.

- Bigger context window ≠ always better quality
- Long context = exponentially more expensive
- Most important information should be near the end (recency bias)
- Summarize old context instead of keeping raw history
- Use Retrieval-Augmented Generation (RAG) for very long documents

MODULE 4 — LLM Behavior & Limitations

Duration: ~45 minutes | Focus: Understanding failure modes

1. Non-Determinism

LLMs do not produce the same output every time, even for identical inputs. This is by design — the model samples from a probability distribution.

```
Same prompt: "Write a haiku about rain"

Run 1: Drops fall silently / On the grey pavement below / Earth drinks its
fill
Run 2: Petrichor rising / Thunder echoes far away / The garden revives
Run 3: Grey skies weeping soft / Rivers swell with borrowed tears / Seeds
begin to wake

→ Three completely different valid responses from the same input
```

2. Sensitivity to Wording

Small changes in how a question is phrased can lead to very different answers. This is why prompt engineering matters so much.

Prompt Version	Effect on Output
"Tell me about Python"	General overview, history, uses
"Explain Python for a 10-year-old"	Simple analogies, no technical jargon
"What are Python's weaknesses?"	Focuses on negatives, criticism
"Python vs Java: which is better?"	Comparative analysis, may hedge
"Write Python code to sort a list"	Code output, minimal explanation

3. No Real Understanding — Pattern Matching

LLMs perform sophisticated pattern matching, not true logical reasoning. This distinction matters for developers.

- Can appear to 'understand' context, but it's statistical association
- May fail at novel problems that weren't in training data
- Arithmetic errors are common — not wired for pure math
- Logical chains can break down over many steps

- Understanding is an illusion produced by vast pattern recognition

4. Recency Bias

ɹ Important: LLMs pay more attention to text that appears near the end of the context. Earlier information can be 'forgotten' or underweighted even if still in context.

- Place your most important instructions at the END of your prompt
- Repeat critical constraints near the user query
- 'Lost in the middle' phenomenon: info in the center of long context is underweighted

5. Common Developer Mistakes

Ӟ Avoid These Pitfalls: These are the most common mistakes made when building with LLMs

- Expecting 100% accuracy — LLMs are probabilistic, not deterministic
- Ignoring token limits — forgetting system prompt + history consumes tokens
- Sending too much irrelevant context — noise hurts quality
- Not handling non-determinism — assuming output will always be identical
- Treating the model like a database — it cannot reliably 'recall' specific facts
- Ignoring hallucination risk — especially critical in medical, legal, financial domains

MODULE 5 — LLM Parameters (Temperature, Top-P, Top-K & More)

Duration: ~60 minutes | This module is key for API usage on Day 2

When you call an LLM via API, you can control how the model generates text using several parameters. Understanding these parameters is essential for getting reliable, high-quality outputs from your applications.

1. Temperature

ɹ Definition: Temperature controls the randomness of the model's output. It scales the probability distribution before sampling.

How it works: After the model calculates probabilities for the next token, temperature is applied as a divisor. Higher temperature = flatter distribution = more random choices. Lower temperature = sharper distribution = more predictable choices.

```
Temperature Examples (prompt: 'The sky is...'):

Temperature = 0.0 (Deterministic):
  → Always: "blue"  (highest probability token every time)

Temperature = 0.7 (Balanced - Default):
  → Sometimes: "blue", "clear", "overcast", "grey"

Temperature = 1.5 (High Creativity):
  → Might say: "dreaming", "singing", "a forgotten canvas"

Temperature = 2.0 (Very High - Often incoherent):
  → Could say: "whispering 47", "absolutely melancholic purple"
```

Temperature Range	Best Use Cases
0.0	Fact extraction, deterministic tasks, testing, code generation where correctness matters
0.1 – 0.3	Technical writing, structured data extraction, classification, reliable Q&A
0.5 – 0.7	General conversation, summarization, balanced responses (most API defaults)
0.8 – 1.0	Creative writing, brainstorming, varied suggestions, content generation

Temperature Range	Best Use Cases
1.0 – 1.5	Highly creative tasks, poetry, experimental ideation, artistic content
1.5+	Avoid in production — outputs become incoherent and unreliable

Important: Temperature = 0 does NOT guarantee identical outputs in all APIs. Some use a small epsilon to prevent true zeros. Always test.

2. Top-P (Nucleus Sampling)

Definition: Top-P limits sampling to the smallest set of tokens whose cumulative probability exceeds P. Also called 'nucleus sampling'.

How it works: The model ranks all possible next tokens by probability. Top-P then cuts off at a cumulative probability threshold, creating a 'nucleus' of likely tokens to sample from.

```
Top-P = 0.9 Example:

Token probabilities after ranking:
  'blue'      → 45%   (cumulative: 45%)
  'clear'     → 25%   (cumulative: 70%)
  'grey'      → 12%   (cumulative: 82%)
  'cloudy'    →  8%   (cumulative: 90%) ← CUT HERE with Top-P=0.90
  'stormy'    →  5%   (cumulative: 95%) ← EXCLUDED
  ... 50,000 other tokens...            ← ALL EXCLUDED

Result: Only 4 tokens are candidates for sampling.
```

Top-P Value	Behavior
0.1	Very restrictive — only the top few tokens considered. Very predictable.
0.5	Moderately restrictive — good balance for factual tasks
0.9	Default for most APIs — broad nucleus, varied but coherent outputs
0.95	Slightly more variety — good for creative tasks
1.0	All tokens considered (effectively disabled) — maximum randomness

Best Practice: Use either Temperature OR Top-P, not both. Adjusting both simultaneously makes behavior unpredictable. Most practitioners prefer Temperature for simplicity.

3. Top-K

Definition: Top-K limits the sampling pool to exactly the K highest-probability tokens, regardless of their actual probability values.

How it works: After ranking all tokens by probability, Top-K simply keeps only the K tokens at the top of the list. Unlike Top-P, the size of the candidate pool is fixed, not dynamic.

```
Top-K = 3 Example:

All possible next tokens ranked:
  Rank 1: 'blue'      → 45%     KEPT
  Rank 2: 'clear'     → 25%     KEPT
  Rank 3: 'grey'      → 12%     KEPT
  Rank 4: 'cloudy'    →  8%     EXCLUDED
  Rank 5: 'stormy'    →  5%     EXCLUDED
  ...(remaining 49,995 tokens)    EXCLUDED

The model can only pick from 'blue', 'clear', or 'grey'.
```

Top-K Value	Behavior
1	Greedy decoding — always picks the single highest probability token
10–20	Very conservative — reliable, low-creativity outputs
40–50	Common default (used by Google/Gemini APIs) — balanced
100+	More variety allowed, approaches unconstrained sampling
Very large	Effectively disables Top-K filtering

Top-P vs Top-K — Key Differences:

Top-P (Nucleus Sampling)	Top-K
Dynamic pool size based on probability	Fixed pool size regardless of probabilities

Top-P (Nucleus Sampling)	Top-K
mass	
Adapts to context (some situations have clear winners)	Rigid — always K candidates
More commonly used in modern APIs (GPT, Claude)	Common in older systems and Gemini
Better handles contexts with high/low certainty	Simpler to reason about

4. Max Tokens (Max Output Length)

ŗ Definition: Max Tokens sets the maximum number of tokens the model can generate in its response. It is a hard cap on output length.

This is different from the context window. Max tokens only limits OUTPUT length, not how much input you can send. The total context = input tokens + max_tokens output.

```
API Example (Python):

response = openai.chat.completions.create(
    model='gpt-4o',
    messages=[{"role": "user", "content": "Explain quantum physics"}],
    max_tokens=150,    # Response will stop at 150 tokens
    temperature=0.7
)
```

Max Tokens Setting	Appropriate For
50–100 tokens	Short answers, classifications, yes/no responses
150–300 tokens	Brief explanations, summaries, short emails
500–800 tokens	Detailed explanations, medium-length content
1000–2000 tokens	Essays, detailed reports, code with comments
4000+ tokens	Long-form content, full documents, complex code

Õ Cost Tip: You are billed for tokens GENERATED, not max_tokens set. Setting max_tokens=4000 but getting a 200-token response only bills 200 output tokens.

5. Frequency Penalty & Presence Penalty

∫ **These parameters** reduce repetition in model outputs. Especially useful for long-form content generation.

Frequency Penalty:

Reduces the likelihood of tokens that have already appeared in the output, proportionally to how many times they've appeared. Higher value = stronger penalty for repeating words.

- Range: -2.0 to 2.0 (0 = disabled, positive = penalizes repetition)
- Use case: Long articles, stories where word variety is important
- Caution: Too high causes awkward vocabulary avoidance

Presence Penalty:

Applies a flat penalty for any token that has appeared at all in the output, regardless of frequency. Encourages the model to introduce new topics.

- Range: -2.0 to 2.0 (0 = disabled)
- Use case: Brainstorming, ensuring topic diversity
- Difference from frequency: flat penalty (appeared once or ten times = same penalty)

6. Stop Sequences

ğ **Definition:** Stop sequences are strings that, when generated by the model, cause it to immediately stop generating further text.

```
API Example with Stop Sequences:

response = openai.chat.completions.create(
    model='gpt-4o',
    messages=[{"role": "user", "content": prompt}],
    stop=["###", "END", "\n\n"],  # Stop at these sequences
    max_tokens=500
)

Use cases:
- Stop at section dividers in structured output
- Stop at newlines for single-line responses
- Stop at custom delimiters in templated output
```

7. Seed (Reproducibility)

Some APIs (like OpenAI) support a seed parameter for reproducible outputs. When the same seed + same temperature + same input are used, outputs are more likely to be identical.

```
# Using seed for reproducibility (OpenAI API):
response = openai.chat.completions.create(
    model='gpt-4o',
    messages=[{"role": "user", "content": "Tell me a fun fact"}],
    seed=42,           # Fixed seed for reproducibility
    temperature=0.7
)

# Note: Same seed doesn't guarantee 100% identical output,
# but significantly increases reproducibility.
```

8. Complete Parameters Reference Table

Parameter	Default	Range	Controls
temperature	1.0	0.0 – 2.0	Randomness / creativity of output
top_p	1.0	0.0 – 1.0	Nucleus sampling probability cutoff
top_k	—	1 – vocab size	Hard limit on token candidate pool
max_tokens	model max	1 – context limit	Maximum length of response
frequency_penalty	0.0	-2.0 – 2.0	Penalty for repeating same words
presence_penalty	0.0	-2.0 – 2.0	Penalty for any repeated topic
stop	null	Array of strings	Custom stop sequences
seed	null	Any integer	Reproducibility of outputs
n	1	1 – 128	Number of responses to generate
stream	false	true / false	Stream tokens as they're generated

9. Parameter Combinations — Practical Recipes

´**Recipes:** Use these parameter presets as starting points for different applications.

Recipe 1: Coding Assistant

```
temperature = 0.1   # Near-deterministic: correctness over creativity
top_p       = 0.95  # Broad but still focused
max_tokens  = 2000  # Allow space for full code blocks
stop        = []    # No custom stops needed
```

Recipe 2: Creative Story Writing

```
temperature = 0.9   # High creativity
top_p       = 0.95  # Broad nucleus
max_tokens  = 1500  # Long-form output
presence_penalty = 0.6  # Encourage topic variety
```

Recipe 3: Customer Support Bot

```
temperature = 0.3   # Consistent, reliable answers
top_p       = 0.9   # Standard nucleus
max_tokens  = 300   # Concise responses
frequency_penalty = 0.3  # Avoid robotic repetition
```

Recipe 4: Data Extraction / JSON Output

```
temperature = 0.0   # Fully deterministic
top_p       = 1.0   # Default (temp=0 overrides this)
max_tokens  = 500   # Enough for structured data
stop        = ['}\n']  # Stop after JSON closes
```

10. How Parameters Interact — Visual Mental Model

```
PROBABILITY DISTRIBUTION FOR NEXT TOKEN:

Without any parameters (raw logits):
blue|└────────────────────┘ 45%
clear|└──────────────┘ 25%
grey|└──────┘ 12%
cloudy|└────┘ 8%
stormy|└──┘ 5%
...(49,995 more tokens)...

After Temperature = 0.3 (sharper):
blue|└────────────────────────────┘ 78%
clear|└────┘ 15%
grey| 5%
cloudy| 2%
(others nearly zero)
```

```
After Temperature = 1.5 (flatter):
blue|▒▒▒▒▒▒▒▒ 22%
clear|▒▒▒▒▒▒▒ 18%
grey|▒▒▒▒▒▒▒ 17%
cloudy|▒▒▒▒▒▒ 15%
stormy|▒▒▒▒▒ 13%
(many tokens become viable)
```

MODULE 6 — Wrap-Up Hands-On Integration

Duration: ~45 minutes | Pull everything together with experiments

Activity: Test the Model

These four tasks apply everything from Modules 1–5. Work through each systematically and observe how the concepts play out in real interactions.

Task 1 — Test Factual Accuracy

20. Ask a factual question (e.g., "What is the capital of Australia?")
21. Verify the answer independently
22. Now ask an obscure question (e.g., "Who won a made-up award in 2019?")
23. Observe hallucination: model answers confidently despite no real answer

Task 2 — Force Hallucination

24. Ask about a topic that doesn't exist
25. Use authoritative-sounding framing: "Describe the XYZ theorem"
26. Notice: Model will fabricate a plausible-sounding explanation
27. This demonstrates probability ≠ truth

Task 3 — Test Context Memory

28. State a specific fact: "My secret code word is MANGO"
29. Have a long conversation or paste a lot of text
30. Ask: "What is my secret code word?"
31. Observe forgetting behavior and context window limitations

Task 4 — Measure Token Impact

32. Run the same question with minimal vs. verbose system prompt
33. Use a token counting tool to compare costs
34. Experiment with temperature 0 vs 1.5 on a creative prompt
35. Observe how parameters change output quality and consistency

Teaching Questions to Ask Students

- "Why did the model fail here? Was it a token issue, context issue, or hallucination?"
- "What parameter would you change to get a more consistent response?"
- "Was this context-related or probability-related behavior?"

- "How would you design a prompt to reduce this failure mode?"
- "What temperature would you use for this use case and why?"

Final Reinforcement — The 3 Pillars

Probability Machine	Tokens = Cost + Limits	Context = Memory
LLM = Next token prediction. Confident ≠ correct. Always verify critical outputs.	Every character costs tokens. Optimize prompts. Monitor usage at scale.	Fixed window. Old info drops. Place critical info near the end.

Transition to Day 2

Й Coming Up: "Now that you understand how LLMs behave and how to control them with parameters — tomorrow we'll use the API to build real applications."

- Day 2 Topic: API calls, prompt engineering, system messages
- Day 2 Topic: Building with OpenAI / Anthropic / other LLM APIs
- Day 2 Topic: Function calling, tool use, structured outputs
- Day 2 Topic: Building your first LLM-powered application

Quick Reference — Parameter Cheat Sheet

Parameter	Low Value	High Value	Use When
Temperature	Deterministic, focused	Random, creative	Low: code/facts High: creative writing
Top-P	Only top tokens	All tokens eligible	Low: factual High: varied outputs
Top-K	Tight candidate pool	Wide candidate pool	Low: technical High: creative
Max Tokens	Short responses	Long responses	Match expected output length
Freq Penalty	Allow repetition	Force word variety	High for long-form prose
Presence Penalty	Stay on topic	Explore new topics	High for brainstorming